



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

Foundations of Algorithms

NP-Completeness – NP-Completeness and Reducibility

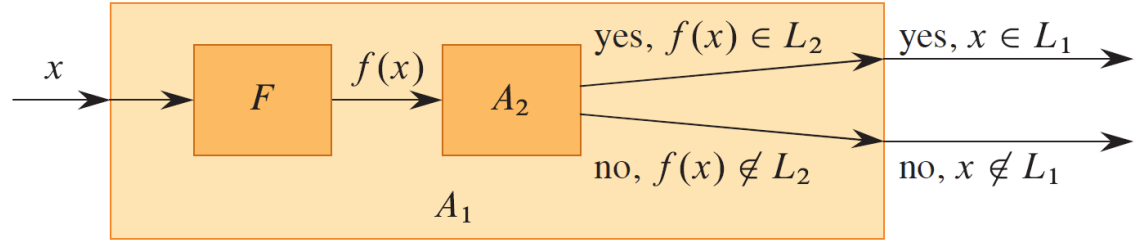
NP-Completeness & Reducibility

- Why NP-Completeness Matters: the Big Question: Is $P \neq NP$?
- If we find a polynomial-time algorithm for any NP-complete problem,
 - $\rightarrow$ then every problem in NP can be solved efficiently
 - $\rightarrow$ That would mean $P = NP$
- But so far... No polynomial-time algorithm ever been found for any NP-complete problem
- Example: HAM-CYCLE
- The Hamiltonian Cycle problem asks whether a graph contains a simple cycle that visits every vertex exactly once
 - It is NP-complete
 - So, if we could solve HAM-CYCLE in polynomial time, we could solve all NP problems that way
- This Section
 - Introduces how to compare problem "hardness" using polynomial-time reductions
 - Shows how to prove a problem is NP-complete using known examples like CIRCUIT-SAT

Reducibility

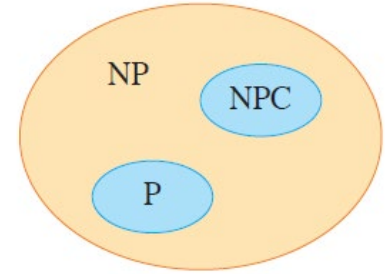
- A problem A is **polynomial-time reducible** to prob B (written $A \leq_p B$)
 - $\rightarrow$ if we can transform any instance of A into an instance of B in polynomial time
 - $\rightarrow$ such that solving B gives us a solution to A
- **If B** is solvable in polynomial time and $A \leq_p B$, **then A** is also solvable in polynomial time
 - Used to compare difficulty between problems
 - A core tool for proving NP-completeness
- How It Works in Practice: to prove problem C is NP-complete:
 - Pick a known NP-complete problem (like SAT)
 - Show $SAT \leq_p C$
 - $\rightarrow$ Build a polynomial-time algorithm that converts any SAT instance to an instance of C
 - This proves that C is at least as hard as SAT

Lemma 34.3



- Lemma 34.3 – Reductions Preserve Polynomial-Time Solvability
- Formal Statement
 - If $L_1 \leq_p L_2$ and $L_2 \in P$, then $L_1 \in P$
- What This Means: If you can reduce L_1 to L_2 in polynomial time, and L_2 is solvable in polynomial time, then L_1 is also solvable in polynomial time
- Why This Matters: Shows that polynomial-time reductions preserve tractability
- If you reduce a problem to something already in P, you've effectively proven your problem is also in P
- **Reduction + efficient solution = efficient** solution for the original problem

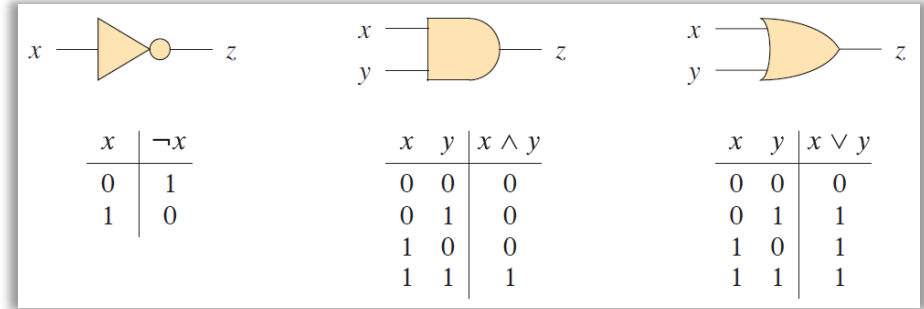
NP-Completeness; Theorem 34.4



- Theorem Statement: If
 - $L \subseteq \{0,1\}^* \in \text{NP}$, and
 - $L' \leq_p L$ for some known NP-complete language L' , then L is NP-complete
- To prove a problem is NP-complete, you just need to
 - Show it's in NP (i.e., solutions can be verified in polynomial time)
 - Show a known NP-complete problem reduces to it in polynomial time
- If a lang L satisfies property 2, but not necessarily property 1, then that L is NP-hard
- This theorem gives a template for proving new problems are NP-complete
 - Once a single problem (like SAT) was shown to be NP-complete, we could use it to prove the NP-completeness of other problems
 - Reductions act like a **hardness chain** — if you can reduce from a known NP-complete problem to your own, your problem must be at least as hard.

Circuit Satisfiability - Definitions

- **CIRCUIT-SAT (Circuit Satisfiability):**
The problem of determining whether a given Boolean combinational circuit has some input that causes the output to be true (i.e., the circuit evaluates to 1)
- **Boolean Circuit:** A finite, acyclic directed graph composed of:
 - Input nodes (variables)
 - Logic gates (AND, OR, NOT)
- **Single output node:** The circuit computes a Boolean function from the input variables

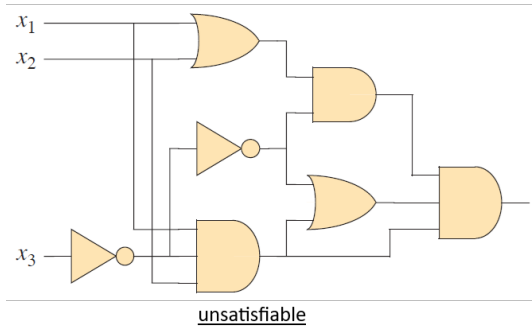
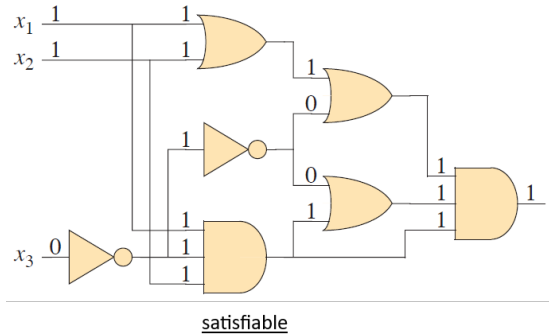


- **Combinational Circuit:** A type of Boolean circuit with no feedback loops—its output depends only on the current inputs, not on any internal state or memory

Circuit Satisfiability (cont)

- Satisfiable Circuit: A Boolean circuit is satisfiable if there exists at least one input assignment that makes the output true (1)
- CIRCUIT-SAT $\in$ NP: CIRCUIT-SAT is in the class NP because: A "yes" instance can be verified in polynomial time and the input assignment serves as a certificate
- **Theorem:** CIRCUIT-SAT is NP-complete
- This is the first problem proven NP-complete (**Cook-Levin Theorem**)
- It serves as the starting point for proving NP-completeness via reductions
- Key Takeaway
 - A circuit is **satisfiable** if and only if there is **some input** that **turns on the output**

Circuit Satisfiability - Example



- Top Circuit: **Satisfiable**
 - Inputs: $x_1 = 1$, $x_2 = 1$, $x_3 = 0$
 - These inputs, the entire circuit evaluates to 1 (true)
 - So, the circuit is satisfiable
- Bottom Circuit: **Unsatisfiable**
 - Same structure, but no input combination causes the output to be true
 - All paths eventually resolve to 0 (false)
 - Therefore, the circuit is unsatisfiable
- Demonstrates what it means for a Boolean circuit to "have a solution"
- Reinforces how logic gates + input configuration affect satisfiability

Lemma 34.5 - CIRCUIT-SAT $\in$ NP

- Lemma 34.5 – Lemma Statement: CIRCUIT-SAT $\in$ NP
 - The circuit-satisfiability problem belongs to the class NP
 - A "yes" instance of CIRCUIT-SAT can be verified in polynomial time
 - The certificate is the input assignment that makes the circuit output 1
 - Simply evaluate the circuit using that input, which takes polynomial time
- Why This Matters in Chapter 34:
 - This lemma is the first step in proving that CIRCUIT-SAT is NP-complete
 - It satisfies the first condition of Theorem 34.4: CIRCUIT-SAT $\in$ NP
 - The next step is to prove that all problems in NP reduce to CIRCUIT-SAT, that's the heart of the Cook-Levin Theorem
- Verifiability is the gateway to NP: if a solution can be quickly checked, the problem belongs in NP

Lemma 34.6 - CIRCUIT-SAT is NP-Hard

- Every language in NP is polynomial-time reducible to CIRCUIT-SAT; i.e., CIRCUIT-SAT is NP-hard
- For any problem in NP, we can build a Boolean circuit that mimics its verifier
- That circuit outputs 1 if and only if the verifier accepts the input
- So, solving CIRCUIT-SAT lets us solve any problem in NP via reduction
- This is the **second key piece** in proving that CIRCUIT-SAT is NP-complete
- Combined with Lemma 34.5 ($\text{CIRCUIT-SAT} \in \text{NP}$), it satisfies both conditions of Theorem 34.4
 - $\text{CIRCUIT-SAT} \in \text{NP}$
 - All NP problems reduce to CIRCUIT-SAT
 - Therefore, CIRCUIT-SAT is NP-complete (Cook-Levin Theorem)
- Key Insight:
 - CIRCUIT-SAT is not just in NP—it's as hard as anything in NP
 - It forms the **starting point** for proving other problems are NP-complete

Thm 34.7 - CIRCUIT-SAT is NP-Complete

- Statement: The circuit-satisfiability problem is NP-complete
- Proof Summary:
 - Follows immediately from:
 - Lemma 34.5: CIRCUIT-SAT $\in$ NP
 - Lemma 34.6: CIRCUIT-SAT is NP-hard
 - Together, they satisfy both conditions of Theorem 34.4 (NP-completeness criterion)
- This is the **root** of the **NP-completeness tree**
- It formally establishes CIRCUIT-SAT as the first NP-complete problem
- This anchors the entire method of proving other problems NP-complete by reduction from CIRCUIT-SAT
- Launches the Cook-Levin Theorem, which is foundational in complexity theory
- Once CIRCUIT-SAT is shown NP-complete, the rest of the chapter builds outward by chaining reductions

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms. MIT press.



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

© The Johns Hopkins University 2025, All Rights Reserved.